

Sistema de Inventario para Ferretería

Plan de Proyecto

Información General

Campo	Detalle
Proyecto	Sistema de Inventario para Ferretería
Tipo	Académico — Desarrollo en Equipo
Stack	NestJS + Next.js + PostgreSQL + Docker + Prisma
Integrantes	Isabella Ramos Betancourt · Julián Andrés Solano Ledesma
Grupo	Grupo 10
Repositorio	github.com/Isarb-21/inventario-ferreteria-sistema

Casos de Uso del Proyecto

#	Caso de Uso
CU-01	Registrar producto con nombre, código, categoría, precio de compra, precio de venta y stock.
CU-02	Gestionar proveedores con datos de contacto y productos que suministran.
CU-03	Registrar compra a proveedor con detalle de productos y actualizar stock automáticamente.
CU-04	Registrar venta al público con detalle de productos, cantidades y total.
CU-05	Consultar productos con stock bajo el mínimo y generar alerta de reabastecimiento.

Plan de Releases

Release 1 — Segundo Corte: Backend y Frontend

Objetivo: Entregar la API REST completa con arquitectura en capas (Controller → Service → Repository) y el frontend con las vistas de CRUD para todas las entidades base.

Sprint	Período	Festivos	Alcance	HUs
Sprint 1 Infraestructura y entidades base	Mar 31 → Abr 11	Abr 2 (Jueves Santo) Abr 3 (Viernes Santo)	Docker Compose, Prisma schema, migraciones, módulos CRUD de Categoría, Producto y Proveedor	HU-01, HU-02, HU-03

Sprint 2 Relaciones, módulos avanzados y Frontend base	Abr 13 → Abr 14	—	Asociación Proveedor-Producto (muchos a muchos), Common module (Filters, Pipes, Interceptors), Frontend: estructura Next.js, listados y formularios de entidades base	HU-04
---	-----------------	---	---	-------

■ Fecha límite entrega Sprint 2: 14 de Abril de 2026 | Sustentación de avances: 15 de Abril de 2026 | Cierre Segundo Corte: 17 de Abril de 2026

Release 2 — Tercer Corte: Integración y Despliegue

Objetivo: Integración completa frontend ↔ backend, formularios avanzados con relaciones, registro de compras y ventas desde la interfaz y despliegue funcional con Docker.

Sprint	Período	Festivos	Alcance	HUs
Sprint 3 Compras, Ventas y Frontend avanzado	Abr 16 → May 8	May 1 (Día del Trabajo)	Módulos de Compra y Venta con transacciones atómicas (prisma.\$transaction), actualización automática de stock, historial con paginación. Frontend: formularios con relaciones (selects dinámicos), páginas de detalle, navegación completa, estados de carga y error.	HU-05, HU-06, HU-07, HU-08
Sprint 4 Alertas, integración final y despliegue	May 11 → May 18	May 18 (Día de la Ascensión)	Alerta de stock bajo mínimo, integración de flujos completos (comprar → actualizar stock → vender → alertar), pruebas de integración, despliegue con Docker Compose, README.	HU-09

■ Fecha límite entrega proyecto: 18 de Mayo de 2026 | Cierre Tercer Corte: 22 de Mayo de 2026

Historias de Usuario

HU-01 — Registro de Productos (CU-01)

Como administrador de la ferretería, quiero registrar nuevos productos con toda su información relevante, para mantener un catálogo actualizado y controlado del inventario.

Criterios de Aceptación

- [] Se puede crear un producto con: nombre, código, categoría, precio de compra, precio de venta y stock inicial.
- [] El código del producto es único en el sistema.
- [] El precio de venta debe ser mayor o igual al precio de compra.
- [] El stock inicial no puede ser un valor negativo.
- [] La categoría debe seleccionarse de una lista existente en el sistema.
- [] Se puede consultar la lista de productos con paginación.
- [] Se puede consultar un producto por ID con todos sus datos completos.
- [] Se puede editar cualquier campo del producto.
- [] Se puede eliminar un producto que no tenga ventas o compras asociadas.
- [] Si se intenta eliminar un producto con movimientos, retorna error descriptivo.

Tareas Técnicas

Backend

- Backend: CreateProductoDto, UpdateProductoDto con class-validator (@IsString, @IsInt, @Min)
- Backend: ProductoRepository (Prisma queries: findAll, findOne, create, update, delete)
- Backend: ProductoService (validaciones de precio, stock y dependencias)
- Backend: ProductoController (endpoints GET /productos, GET /productos/:id, POST, PUT/:id, DELETE/:id)

Frontend

- Frontend: Página de listado app/productos/page.tsx
- Frontend: Formulario de creación app/productos/new/page.tsx
- Frontend: Página de detalle y edición app/productos/[id]/page.tsx
- Frontend: services/producto.service.ts — llamadas a la API usando lib/api.ts

HU-02 — Gestión de Categorías de Productos (CU-01)

Como administrador de la ferretería, quiero gestionar las categorías del catálogo, para organizar los productos de forma clara y facilitar su consulta y búsqueda.

Criterios de Aceptación

- [] Se puede crear una categoría con nombre y descripción.
- [] El nombre de la categoría es único en el sistema.
- [] Se puede listar todas las categorías disponibles.

- [] Se puede editar el nombre y descripción de una categoría.
- [] Se puede eliminar una categoría que no tenga productos asociados.
- [] Si se intenta eliminar una categoría con productos, retorna error descriptivo.
- [] Al registrar o editar un producto, las categorías disponibles se listan dinámicamente.

Tareas Técnicas

Backend

- Backend: CreateCategoriaDto, UpdateCategoriaDto con class-validator
- Backend: CategoriaRepository (Prisma queries: findAll, findOne, create, update, delete)
- Backend: CategoriaService (validación de unicidad y dependencias con productos)
- Backend: CategoriaController (endpoints GET /categorias, GET/:id, POST, PUT/:id, DELETE/:id)

Frontend

- Frontend: Página de listado app/categorias/page.tsx
- Frontend: Formulario de creación app/categorias/new/page.tsx
- Frontend: services/categoria.service.ts — llamadas a la API

HU-03 — Registro de Proveedores (CU-02)

Como administrador de la ferretería, quiero registrar y gestionar los proveedores con sus datos de contacto, para saber a quién acudir para reabastecer cada producto.

Criterios de Aceptación

- [] Se puede crear un proveedor con: nombre, NIT/cédula, teléfono, correo electrónico y dirección.
- [] El NIT/cédula del proveedor es único en el sistema.
- [] El correo electrónico debe tener formato válido.
- [] Se puede consultar la lista de proveedores con paginación.
- [] Se puede consultar un proveedor por ID con sus datos completos y productos asociados.
- [] Se puede editar la información del proveedor.
- [] Se puede eliminar un proveedor sin compras registradas.
- [] Si se intenta eliminar un proveedor con compras, retorna error descriptivo.

Tareas Técnicas

Backend

- Backend: CreateProveedorDto, UpdateProveedorDto con class-validator (@IsEmail, @IsString)
- Backend: ProveedorRepository (Prisma queries con include de productos asociados)
- Backend: ProveedorService (validaciones de unicidad y dependencias con compras)
- Backend: ProveedorController (endpoints GET /proveedores, GET/:id, POST, PUT/:id, DELETE/:id)

Frontend

- Frontend: Página de listado app/proveedores/page.tsx

- Frontend: Formulario de creación `app/proveedores/new/page.tsx`
- Frontend: Página de detalle `app/proveedores/[id]/page.tsx`
- Frontend: `services/proveedor.service.ts` — llamadas a la API

HU-04 — Asociación de Productos a Proveedor (CU-02)

Como administrador de la ferretería, quiero asociar productos específicos a cada proveedor, para saber fácilmente a qué proveedor acudir cuando necesito reabastecer un producto.

Criterios de Aceptación

- [] Al crear o editar un proveedor, se pueden seleccionar múltiples productos que él suministra.
- [] Se puede consultar la lista de productos asociados a un proveedor.
- [] Se puede consultar desde la vista de un producto qué proveedores lo suministran.
- [] Se puede agregar o quitar productos de la lista de un proveedor sin eliminarlo.
- [] Si un producto no tiene proveedor asociado, se muestra una advertencia informativa.

Tareas Técnicas

Backend

- Backend: Prisma schema — tabla intermedia `ProveedorProducto` para relación muchos-a-muchos entre `Proveedor` y `Producto`
- Backend: `ProveedorRepository` — métodos para asociar/desasociar productos (`connect/disconnect` en Prisma)
- Backend: `ProveedorController` — `POST /proveedores/:id/productos`, `DELETE /proveedores/:id/productos/:productId`
- Backend: `ProductoController` — `GET /productos/:id/proveedores`

Frontend

- Frontend: `app/proveedores/[id]/page.tsx` — selector múltiple de productos en la vista de detalle
- Frontend: `components/forms/ProductosSelectorMultiple.tsx` — componente reutilizable de selección múltiple

HU-05 — Registro de Compra a Proveedor (CU-03)

Como administrador de la ferretería, quiero registrar las compras realizadas a proveedores, para llevar un historial de abastecimiento y actualizar el stock automáticamente.

Criterios de Aceptación

- [] Se puede crear una compra con: proveedor, fecha y detalle de productos (cantidad y precio de compra por ítem).
- [] El detalle debe contener al menos un producto.
- [] Las cantidades deben ser valores enteros positivos.
- [] Al confirmar la compra, el stock de cada producto en el detalle se incrementa automáticamente.
- [] Se puede consultar el historial de compras con paginación.
- [] Se puede consultar una compra por ID con su detalle completo.

- [] No se puede editar ni eliminar una compra ya registrada (trazabilidad).
- [] El total de la compra se calcula automáticamente a partir del detalle.

Tareas Técnicas

Backend

- Backend: CreateCompraDto con array de DetalleCompraDto usando @ValidateNested y @Type
- Backend: CompraRepository — usa prisma.\$transaction para crear compra + detalles + actualizar stock atómicamente
- Backend: CompraService — cálculo del total y orquestación de la transacción
- Backend: CompraController (endpoints GET /compras, GET /compras/:id, POST /compras)

Frontend

- Frontend: app/compras/new/page.tsx — formulario con selector de proveedor y tabla dinámica de productos
- Frontend: app/compras/page.tsx — listado de compras con paginación
- Frontend: services/compra.service.ts — llamadas a la API usando lib/api.ts

HU-06 — Consulta de Historial de Compras por Proveedor (CU-03)

Como administrador de la ferretería, quiero consultar todas las compras realizadas a un proveedor específico, para analizar el historial de abastecimiento.

Criterios de Aceptación

- [] Se puede filtrar el listado de compras por proveedor desde un selector.
- [] Los resultados muestran: fecha de compra, productos adquiridos, cantidades y total.
- [] Se puede ordenar el listado por fecha (ascendente y descendente).
- [] Si el proveedor no tiene compras, se muestra un mensaje informativo.

Tareas Técnicas

Backend

- Backend: CompraRepository — query findAll recibe proveedorId? como parámetro opcional
- Backend: CompraController — GET /compras?proveedorId= con query param opcional (@Query)

Frontend

- Frontend: app/compras/page.tsx — selector de proveedor como filtro sobre el listado
- Frontend: components/ui/FiltroProveedor.tsx — componente de selector reutilizable

HU-07 — Registro de Venta al Público (CU-04)

Como cajero o administrador de la ferretería, quiero registrar las ventas realizadas a clientes, para llevar un control de ingresos y actualizar el stock automáticamente.

Criterios de Aceptación

- [] Se puede crear una venta con: fecha y detalle de productos (producto, cantidad y precio de venta).
- [] El detalle debe contener al menos un producto.
- [] No se puede vender una cantidad mayor al stock disponible de un producto.
- [] Al confirmar la venta, el stock de cada producto se reduce automáticamente.
- [] El total de la venta se calcula automáticamente.
- [] Se puede consultar el historial de ventas con paginación.
- [] Se puede consultar una venta por ID con su detalle completo.
- [] No se puede editar ni eliminar una venta ya registrada (trazabilidad).

Tareas Técnicas

Backend

- Backend: CreateVentaDto con array de DetalleVentaDto usando @ValidateNested y @Type
- Backend: VentaRepository — usa prisma.\$transaction: verificar stock + crear venta + detalles + reducir stock atómicamente
- Backend: VentaService — validación de stock disponible y cálculo del total
- Backend: VentaController (endpoints GET /ventas, GET /ventas/:id, POST /ventas)

Frontend

- Frontend: app/ventas/new/page.tsx — formulario con tabla dinámica de productos
- Frontend: app/ventas/[id]/page.tsx — detalle de venta
- Frontend: services/venta.service.ts — llamadas a la API usando lib/api.ts

HU-08 — Consulta de Historial de Ventas (CU-04)

Como administrador de la ferretería, quiero consultar el listado de ventas registradas y ver el detalle de cada una, para revisar qué se ha vendido y verificar los movimientos del día.

Criterios de Aceptación

- [] Se puede consultar la lista de todas las ventas con paginación.
- [] El listado muestra: ID de venta, fecha, cantidad de ítems y total.
- [] Se puede consultar una venta por ID con su detalle completo (productos, cantidades y precios).
- [] Si no hay ventas registradas, se muestra un mensaje informativo.
- [] Se puede acceder al detalle de una venta desde el listado.

Tareas Técnicas

Backend

- Backend: VentaRepository — query findAll con paginación e include de DetalleVenta + Producto
- Backend: VentaRepository — query findOne con include completo del detalle

Frontend

- Frontend: app/ventas/page.tsx — tabla de ventas con enlace al detalle por fila
- Frontend: app/ventas/[id]/page.tsx — tabla de productos del detalle de la venta
- Frontend: services/venta.service.ts — funciones getAll() y getByid(id) usando lib/api.ts

HU-09 — Alerta de Productos con Stock Bajo Mínimo (CU-05)

Como administrador de la ferretería, quiero consultar los productos cuyo stock esté por debajo del mínimo establecido, para tomar decisiones de reabastecimiento antes de quedarme sin existencias.

Criterios de Aceptación

- [] Cada producto tiene un campo stockMinimo que se define al crearlo o editarlo.
- [] Existe una vista que lista únicamente los productos con stock actual menor al stock mínimo.
- [] El listado muestra: nombre, código, categoría, stock actual y stock mínimo.
- [] Si no hay productos con stock bajo, se muestra un mensaje de confirmación positivo.
- [] El listado es de solo lectura; para corregir el stock se debe registrar una compra (HU-05).

Tareas Técnicas

Backend

- Backend: campo stockMinimo Int @default(0) agregado al modelo Producto en prisma/schema.prisma
- Backend: ProductoRepository — método findStockBajo: findMany con where { stock: { lt: stockMinimo } }
- Backend: ProductoService — método getProductosStockBajo() que delega al repositorio
- Backend: ProductoController — endpoint GET /productos/stock-bajo

Frontend

- Frontend: app/productos/stock-bajo/page.tsx — tabla de productos en alerta
- Frontend: components/ui/AlertaBadge.tsx — badge reutilizable visible en app/layout.tsx

Definition of Done (DoD) Global

Cada Historia de Usuario se considera terminada cuando cumple todos los siguientes criterios:

Backend

- ☐ Endpoint(s) implementados con arquitectura en capas: Controller → Service → Repository.
- ☐ DTOs con validaciones usando class-validator y class-transformer.
- ☐ Manejo de errores con excepciones HTTP apropiadas (NotFoundException, ConflictException, BadRequestException).
- ☐ Respuestas con formato uniforme (interceptor aplicado).
- ☐ Endpoint probado manualmente con Postman/Thunder Client y funcionando correctamente.

Frontend

- ☐ Página(s) implementada(s) con componentes reutilizables.
- ☐ Consumo del API a través de la capa de services/.
- ☐ Manejo de estados: carga (loading), éxito y error.
- ☐ Formularios con validación del lado del cliente.
- ☐ Diseño responsivo y navegable.

Infraestructura y Código

- ☐ Código versionado en GitHub con commits descriptivos.
- ☐ El servicio funciona correctamente con docker compose up.
- ☐ No hay errores de consola ni advertencias críticas.
- ☐ Las migraciones de Prisma están aplicadas y el esquema es consistente.

Entidades del Modelo de Datos (Prisma)

Nota: La entidad Proveedor aparece dos veces en la tabla de relaciones porque tiene dos relaciones distintas: una $N \rightarrow M$ con Producto (catálogo de suministro) y una $1 \rightarrow N$ con Compra (transacciones). Prisma no se confunde porque las diferencia internamente por nombres de campo: proveedorProductos[] y compras[].

Entidad Origen	Cardinalidad	Entidad Destino	Descripción
Categoria	$1 \rightarrow N$	Producto	Una categoría contiene múltiples productos.
Proveedor	$N \rightarrow M$	Producto	Un proveedor suministra múltiples productos (tabla ProveedorProducto). Nota: esta relación modela el catálogo de suministro, no transacciones.
Proveedor	$1 \rightarrow N$	Compra	Un proveedor tiene múltiples compras registradas. Nota: aunque el nombre 'Proveedor' se repite, es la misma entidad con dos relaciones distintas — suministro de catálogo y transacciones de compra. Prisma las diferencia internamente por los campos proveedorProductos[] y compras[].
Compra	$1 \rightarrow N$	DetalleCompra	Una compra tiene múltiples ítems de detalle.
Producto	$1 \rightarrow N$	DetalleCompra	Un producto aparece en múltiples detalles de compra.
Venta	$1 \rightarrow N$	DetalleVenta	Una venta tiene múltiples ítems de detalle.
Producto	$1 \rightarrow N$	DetalleVenta	Un producto aparece en múltiples detalles de venta.

Resumen de Entidades

Entidad	Campos principales
Categoria	id, nombre (unique), descripcion
Producto	id, nombre, codigo (unique), categoriald, precioCompra, precioVenta, stock, stockMinimo
Proveedor	id, nombre, nit (unique), telefono, correo, direccion
ProveedorProducto	proveedorId, productold (compound unique)
Compra	id, proveedorId, fecha, total
DetalleCompra	id, comprald, productold, cantidad, precioUnitario
Venta	id, fecha, total
DetalleVenta	id, ventald, productold, cantidad, precioUnitario

Cronograma de Releases

Release 1 — Segundo Corte (Cierre académico: 17 Abr 2026) — Backend + Frontend Base

Fecha límite interna del equipo: 14 de Abril | Sustentación de avances: 15 de Abril de 2026

Sprint	Período	Alcance	Festivos
Sprint 1	Mar 31 → Abr 11	Docker Compose, Prisma schema, Categoría, Producto, Proveedor (CRUDs completos)	Abr 2-3 (Semana Santa)
Sprint 2	Abr 13 → Abr 14	Asociación Proveedor-Producto, Common Module (Filters/Pipes/Interceptors), Frontend: estructura Next.js, listados y formularios base	—

■ Sprint 2 listo: 14 de Abril | Sustentación: 15 de Abril | Cierre Segundo Corte: 17 de Abril de 2026

Release 2 — Tercer Corte (Cierre académico: 22 May 2026) — Integración + Despliegue

Fecha límite interna del equipo: 18 de Mayo de 2026

Sprint	Período	Alcance	Festivos
Sprint 3	Abr 16 → May 8	Compra, Venta con transacciones atómicas, historial con filtros, Frontend avanzado: formularios con selects dinámicos, navegación completa	May 1 (Día del Trabajo)
Sprint 4	May 11 → May 18	Alerta de stock bajo mínimo, integración de flujos completos, pruebas de integración, Docker Compose validación final, README y documentación	May 18 (Ascensión)

■ Proyecto terminado: 18 de Mayo | Cierre Tercer Corte: 22 de Mayo de 2026

Festivos Colombianos 2026 (Marzo — Mayo)

Fecha	Festivo	Sprint afectado
Jueves 2 de Abril	Jueves Santo	Sprint 2
Viernes 3 de Abril	Viernes Santo	Sprint 2
Viernes 1 de Mayo	Día del Trabajo	Sprint 3
Lunes 18 de Mayo	Día de la Ascensión	Sprint 4

Estado del Proyecto

- [x] Plan de releases revisado y aprobado
- [x] Historias de Usuario revisadas y aprobadas
- [x] Criterios de Aceptación revisados y aprobados
- [x] Definition of Done revisado y aprobado
- [x] Modelo de datos revisado y aprobado
- [x] GitHub creado con Issues y Milestones

- **Repositorio:** github.com/lsarb-21/inventario-ferreteria-sistema
- Issues: 9 HUs + 1 DoD (pinned)
- Milestones: 4 sprints con fechas de cierre